

Golf/testes.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <CUnit/CUnit.h>
4  #include "Jogo.h"
5
6  //
7  //
8  // Funções de Teste da Lógica
9  //
10 //
11
12 // Função que testa a validade das jogadas (valores imediatamente acima ou
13 // abaixo)
14 void test_jogadaValida(void) {
15     carta c1 = {5, 0}; ← carta base
16     carta c2 = {6, 1};
17     carta c3 = {4, 2}; } → carta o cima e a baixo
18     carta c4 = {5, 3}; ← carta igual
19     CU_ASSERT(jogadaValida(c1, c2) == 1); } → tanto a carta o cima como
20     CU_ASSERT(jogadaValida(c1, c3) == 1); } → a baixo é válida
21     CU_ASSERT(jogadaValida(c1, c4) == 0); ← não é válida
22 }
23 // Função que verifica se o baralho é criado corretamente com as 52 cartas
24 void test_criarBaralho(void) {
25     jogo j; ← criamos um jogo
26     criarBaralho(&j); ← mandamos criar o baralho
27     CU_ASSERT(j.cartas_baralho == 52); ← o n° de cartas tem de ser 52
28     CU_ASSERT(j.baralho[0].num == 1); ← garantimos que a primeira carta é
29 }
30 }
31 // Função que testa se as cartas são baralhadas mantendo as todas no
32 // baralho
33 void test_baralharCartas(void) {
34     jogo j;
35     criarBaralho(&j);
36     baralharCartas(&j);
37     CU_ASSERT(j.cartas_baralho == 52); ← tem de ser 52 as cartas
38 }
39 // Função que testa a distribuição inicial das cartas pelas 7 pilhas
40 void test_distribuirCartas(void) {
41     jogo j;
42     criarBaralho(&j);
43     distribuirCartas(&j);
44     CU_ASSERT(j.cartas_baralho == 17); ← o baralho tem de ter 17 cartas
45     CU_ASSERT(j.quantidade_pilhas[0] == 5); ← o n° de cartas na

```



```
46 }
47
48 // Função que testa a colocação da primeira carta na pilha de descarte
para iniciar
49 void test_descarte(void) {
50     jogo j;
51     criarBaralho(&j);
52     distribuirCartas(&j);
53     descarte(&j); ← faz o descarte de uma carta do baralho
54     CU_ASSERT(j.cartas_baralho == 16); ← testa se o baralho tem mais
55     CU_ASSERT(j.cartas_descarte == 1); uma carta e o descarte +1
56 }
57
58 // Função que testa o biscoar de cartas com o baralho completo e vazio
59 void test_biscoarCarta(void) { ← testar se o baralho estiver vazio
60     jogo j; de m consegue biscoar
61     j.cartas_baralho = 1;
62     j.cartas_descarte = 0;
63     CU_ASSERT(biscoarCarta(&j) == 1); ← 1º consegue biscoar
64     CU_ASSERT(biscoarCarta(&j) == 0); ← 2º m consegue
65 }
66
67 // Função que testa mover uma carta da pilha para o descarte validando as
regras
68 void test_moverDaPilha(void) {
69     jogo j;
70     j.cartas_descarte = 1; ← cartas para descarte = 1
71     j.descarte[0] = (carta){5, 0}; ← A carta é o 3 de espadas (descarte)
72     j.quantidade_pilhas[0] = 1; ← A pilha 1 só tem 1 carta
73     j.pilhas[0][0] = (carta){6, 1}; ← A carta na pilha 1 é 6 de Copas
74     j.quantidade_pilhas[1] = 1; ← na pilha 2, tem 1 carta
75     j.pilhas[1][0] = (carta){10, 2}; ← a carta é um 10 de ouros
76     CU_ASSERT(moverDaPilha(&j, 1) == 1); ← tem de conseguir mover o 6 para
77     CU_ASSERT(moverDaPilha(&j, 2) == 0); o 6 está no topo logo m pode
78 } mover o 10
79
80 // Função que testa se o jogo reconhece a vitória quando todas as pilhas
ficam vazias
81 void test_verificarVitoria(void) {
82     jogo j;
83     for (int i = 0; i < 7; i++) j.quantidade_pilhas[i] = 5; ← Se as pilhas
84     CU_ASSERT(verificarVitoria(&j) == 0); tem cartas m
85     for (int i = 0; i < 7; i++) j.quantidade_pilhas[i] = 0; tem vitória
86     CU_ASSERT(verificarVitoria(&j) == 1); ← Se tiverem
87 } vezes tem
88 vitória
89 //
90 //
91 // Funções Auxiliares para o CUnit
92 //
```


93 //

94

```
95 // Função que adiciona todos os testes à suite
```

```
96 void adicionarTestes(CU_pSuite suite) {
```

```

97 CU_add_test(suite, "jogadaValida", test_jogadaValida);
98 CU_add_test(suite, "criarBaralho", test_criarBaralho);
99 CU_add_test(suite, "baralharCartas", test_baralharCartas);
00 CU_add_test(suite, "distribuirCartas", test_distribuirCartas);
01 CU_add_test(suite, "descarte", test_descarte);
02 CU_add_test(suite, "biscarCarta", test_biscarCarta);
03 CU_add_test(suite, "moverDaPilha", test_moverDaPilha);
04 CU_add_test(suite, "verificarVitoria", test_verificarVitoria);
05 }

```

Adelma
O Foster

06

```
07 // Função que cria a suíte de testes
```

```
08 int configurarSuite(void) {
09     CU_pSuite suite = CU_add_suite("Suite Logica", NULL, NULL)
```

nao tem nenhum
harddown
naque isolamos os
NULL);

```
10 if (NULL == suite) { ← Se o suite n foi criado
```

ULL); unguis cum
fichio exterior
cum pestes

```
11 CU_cleanup_registry();
```

```
12     return 0;
```

13 Cheia } ou de end a inicia

```
14    adicionarTestes(suite);
```

```
15 return 1;
```

16 }

117

```
118 // Função que verifica na lista do CUnit se um teste específico falou
```

```
119 int verificarSeFalhou(CU_pTest pTest) {
```

```
120 int falhou = 0; // 0 m = 0 falhas, falhas = 0
```

```
121 CU_pFailureRecord pFailure = CU_get_failure_list();
```

```
222 while (pFailure != NULL && falhou == 0) {
223     if (!(*pFailure).nTest || nTest) {
```

```

223 if ((^pFailure).plest == plest) {
224     falhou = 1;

```

24 $\text{radius} = 1;$

```

25 }
26 pFailure = (*pFailure) pNext;

```

practure = (practure).next, also, answer is fraction

```
28 return falhou:
```

129 }

30.

```
131 // Função que imprime o resultado de cada teste individualmente
```

```
132 void imprimirTestesDaSuite(CU_pSuite pSuite) {
```

```
133 CU_pTest pTest = (*pSuite).pTest; // Resolves no hidden tests
```

```
134 while (pTest != NULL) { ← Enqueues broken tasks
```

```
135     if (verificarSeFalhou(pTest) == 1) {
```

```
136     printf("Funcao %s: NAO PASSOU\n", (*pTest).pName);
```

```
137         } else {
```

```
138     printf("Funcao %s: PASSOU\n", (*pTest).pName);
139 }
```

139 }
140 }

```

40     plist = (*plist).pNext;
41 }

```

141 }

Se fabrica
diz que
fabrica

* Send dirz go
person

passa para o próximo teste


```

142 }
143
144 // Função que percorre as suites e inicia a impressão
145 void imprimirResultados(void) {
146     printf("\n");
147     CU_pSuite pSuite = (*CU_get_registry()).pSuite;
148     while (pSuite != NULL) {
149         imprimirTestesDaSuite(pSuite);
150         pSuite = (*pSuite).pNext;
151     }
152     printf("\n");
153 }
154
155 //
156 //
157 // Função Main
158 //
159 //
160
161 int main(void) {
162     if (CUE_SUCCESS != CU_initialize_registry()) {
163         return CU_get_error();
164     }
165
166     if (configurarSuite() == 0) {
167         return CU_get_error();
168     }
169
170     CU_run_all_tests();
171     imprimirResultados();
172
173     CU_cleanup_registry();
174     return CU_get_error();
175 }
176

```

Handwritten notes:

- vai ao registro e pega no suite* (pointing to line 147)
- Enquanto houver testes* (pointing to line 148)
- Passo a suite a seguir para a próxima* (pointing to line 150)
- Para para a próxima suite tem só tem 1 logo para* (pointing to line 152)
- Fizemos isso pois se o resultado mudar os testes continuam a funcionar.*
- Fizemos para testar a arquitetura do CUnit*
- Após inicializar o registro, se não for sucesso do erro* (pointing to line 162)
- Se não conseguir configurar o suite do erro* (pointing to line 166)
- testa* (pointing to line 170)
- Agora imprimimos resultados* (pointing to line 171)
- Apaga os registros na ram para não continuar o oculto RAM do computador* (pointing to line 173)
- Devolvemos o erro caso exista* (pointing to line 174)